

《 Introduction to Embedded System 》

Lab-1

Major: **Software Engineering**

Name: **Md Tasnimul Hasan**

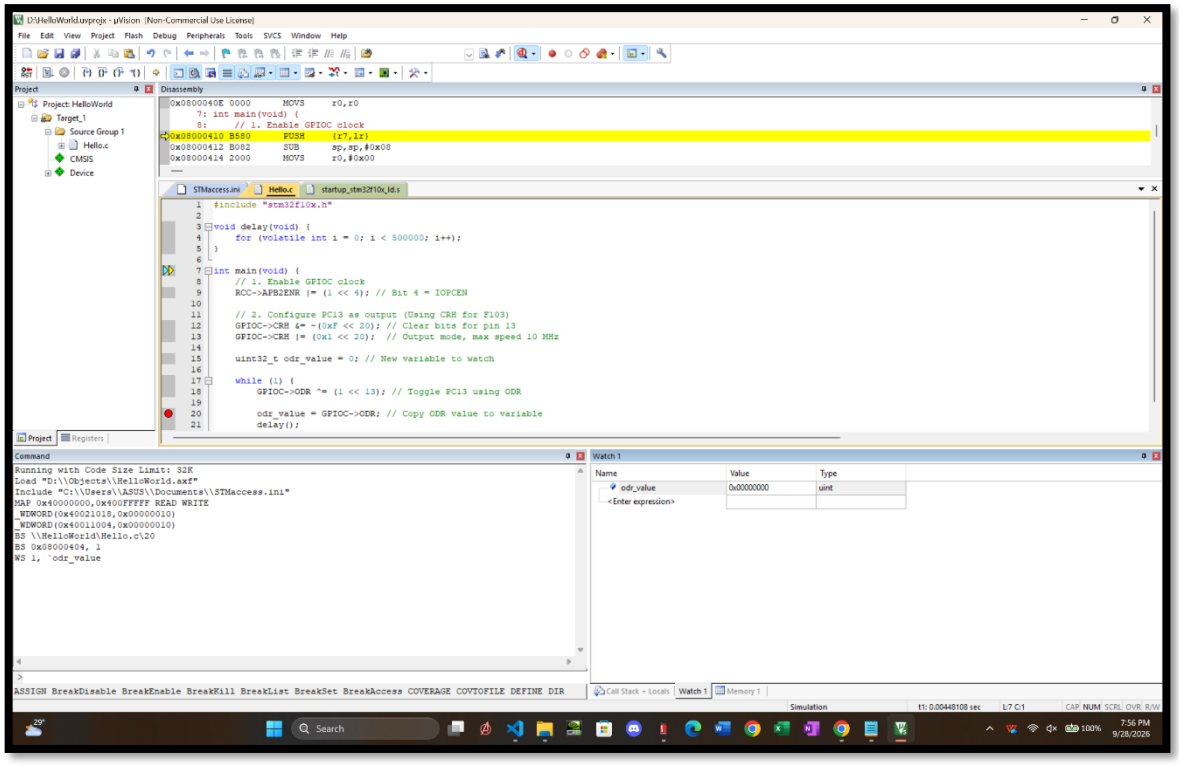
Student ID: **202480090104**

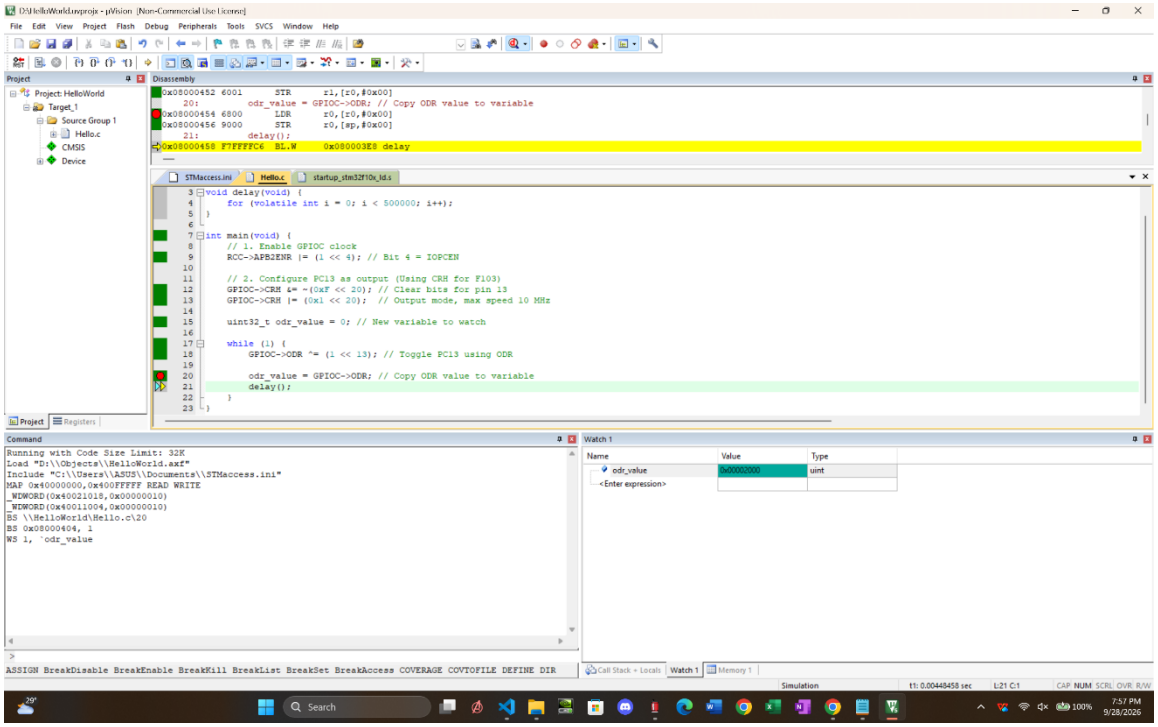
◇ Task 1: LED Blinking Using ODR

Objective:
To learn how to configure a GPIO pin (PC13) as an output using the CRH register and to toggle it periodically using the Output Data Register (ODR).

Description:
In this task, the GPIOC clock was enabled and PC13 was configured as a push-pull output. Inside an infinite loop, the ODR register's 13th bit was toggled using the XOR (^=) operator, and a software delay was used between toggles. A variable output was used to monitor the ODR value in the Watch Window. The value alternated between 0x00002000 (LED ON) and 0x00000000 (LED OFF), successfully simulating an LED blinking pattern.

Output:



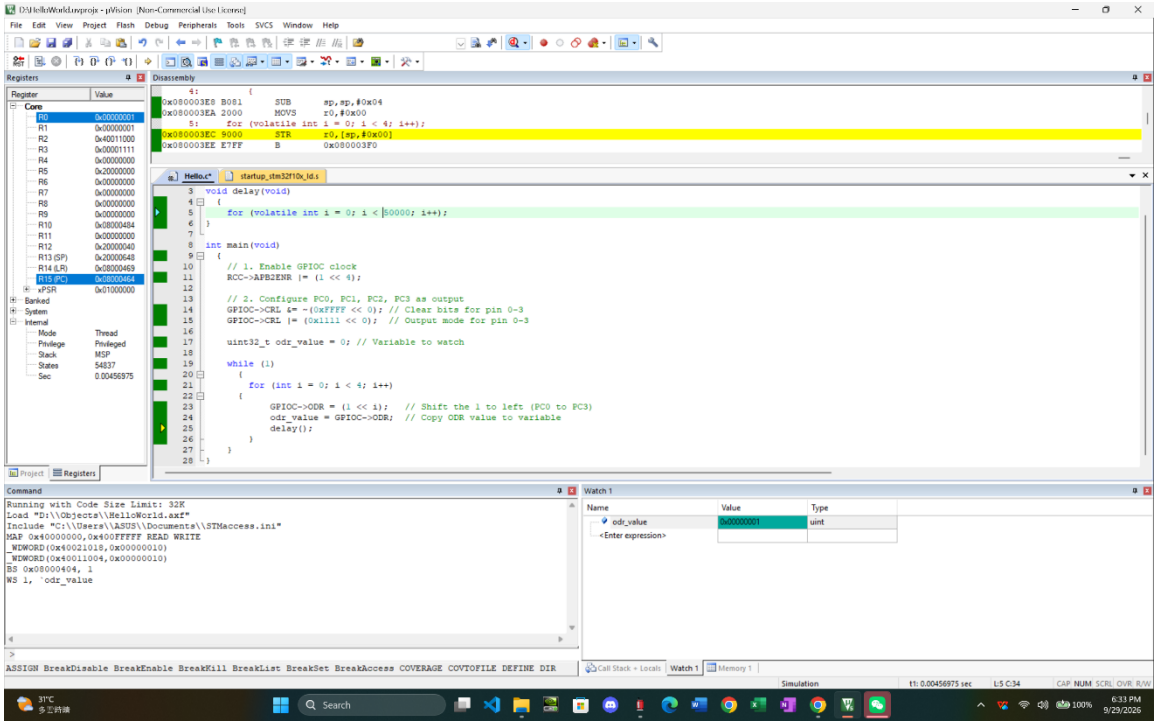


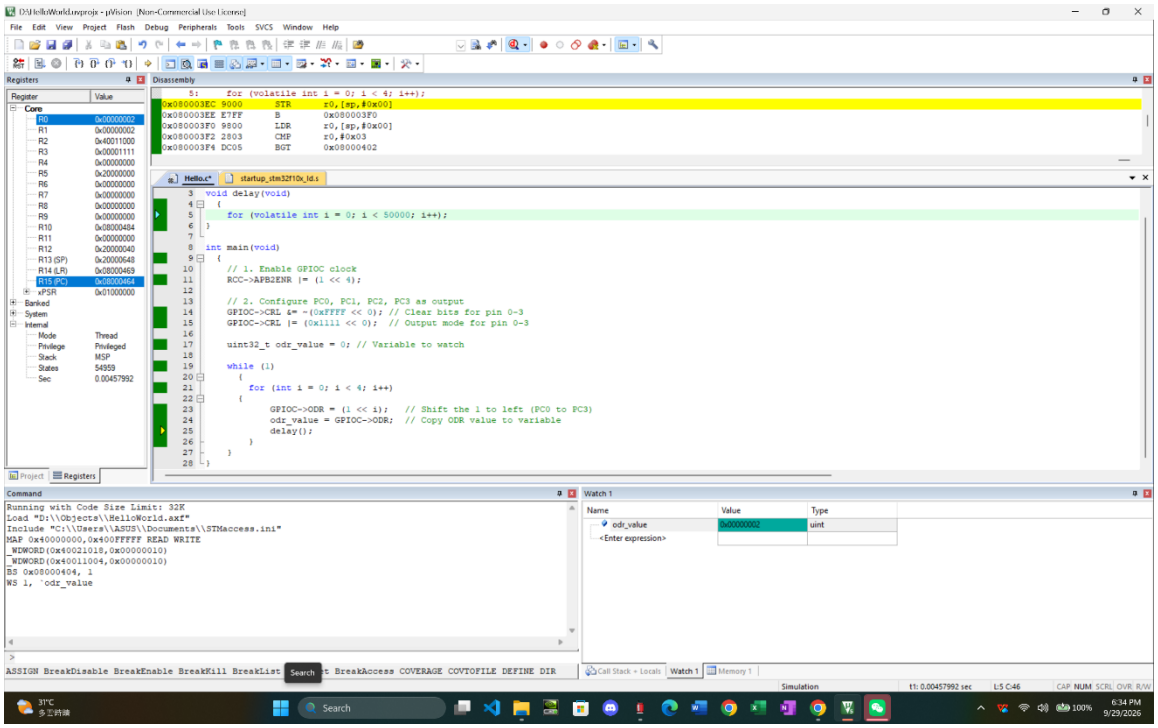
Task 2: Flowing LED Pattern (Running Light)

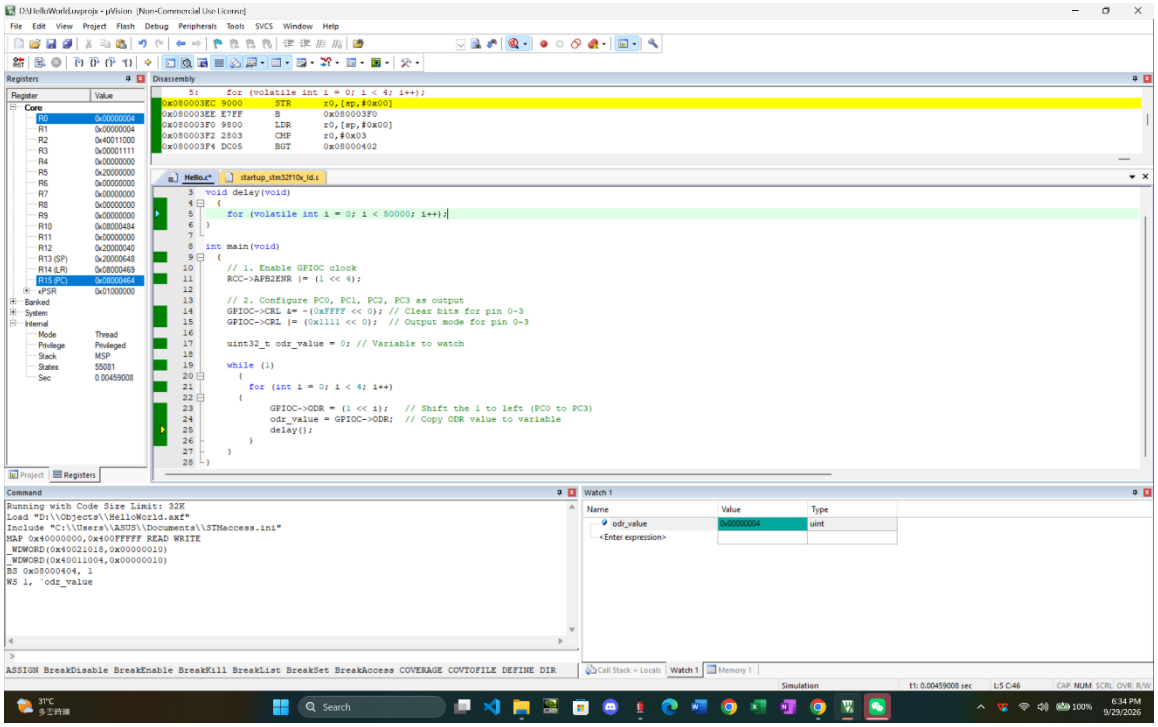
Objective:
To simulate a running LED pattern using multiple GPIO pins (PC0–PC3) by shifting bits through the Output Data Register (ODR).

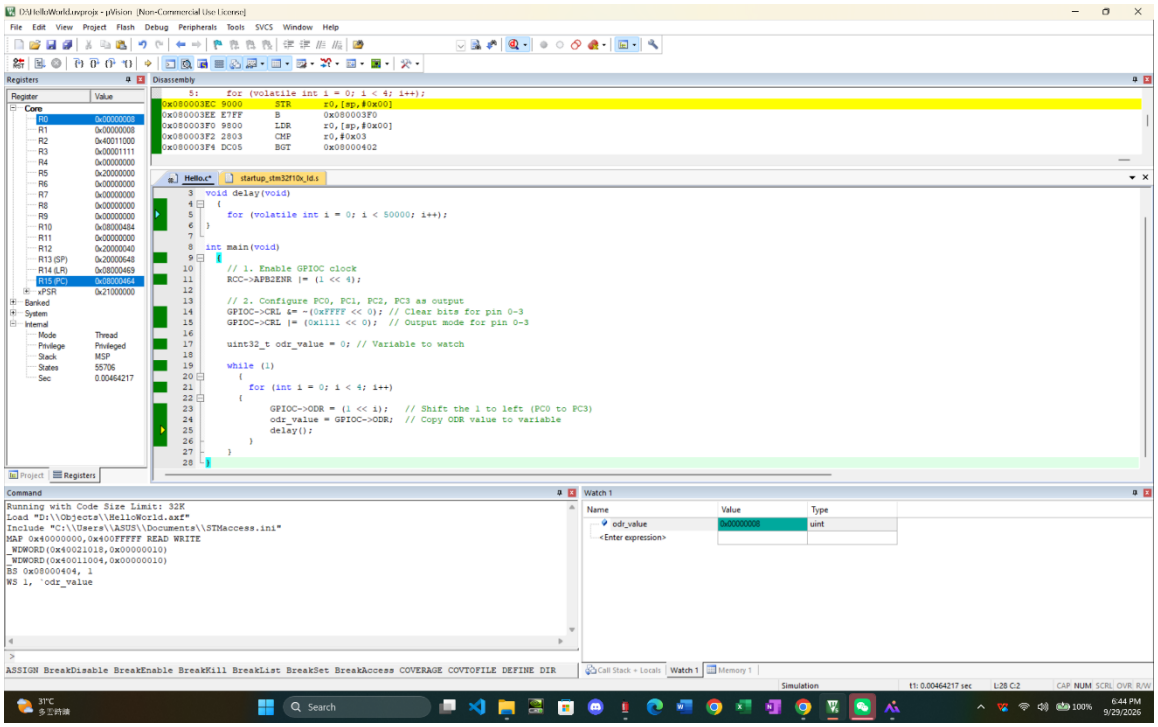
Description:
In this task, PC0, PC1, PC2, and PC3 were configured as output pins. A for loop was used with a bit-shift operation (1 << i) to send a HIGH signal to one pin at a time. The ODR value changed sequentially as 0x00000001 → 0x00000002 → 0x00000004 → 0x00000008, then repeated. This created a flowing LED or running light effect. The variable output was monitored in the Watch Window to verify the bit-shifting behavior.

Output:


File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Disassembly
Hello.c*
startup_stm32f10x_ld.s
void delay(void)
{
for (volatile int i = 0; i < 50000; i++);
}
int main(void)
{
// 1. Enable GPIOC clock
RCC->APB2ENR |= (1 << 4);
// 2. Configure PC0, PC1, PC2, PC3 as output
GPIOC->CRL &= ~(0xFFFF << 0); // Clear bits for pin 0-3
GPIOC->CRL |= (0x1111 << 0); // Output mode for pin 0-3
uint32_t odr_value = 0; // Variable to watch
while (1)
{
for (int i = 0; i < 4; i++)
{
GPIOC->ODR = (1 << i); // Shift the 1 to left (PC0 to PC3)
odr_value = GPIOC->ODR; // Copy ODR value to variable
delay();
}
}
}
Command
Running with Code Size Limit: 32K
Load "D:\\Objects\\HelloWorld.axf"
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x40000000,0x400FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
WS 1, `odr_value
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Watch 1
Name
Value
Type
odr_value
0x00000001
uint
<Enter expression>
Search


File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Disassembly
Hello.c*
startup_stm32f10x_ld.s
void delay(void)
{
for (volatile int i = 0; i < 50000; i++);
}
int main(void)
{
// 1. Enable GPIOC clock
RCC->APB2ENR |= (1 << 4);
// 2. Configure PC0, PC1, PC2, PC3 as output
GPIOC->CRL &= ~(0xFFFF << 0); // Clear bits for pin 0-3
GPIOC->CRL |= (0x1111 << 0); // Output mode for pin 0-3
uint32_t odr_value = 0; // Variable to watch
while (1)
{
for (int i = 0; i < 4; i++)
{
GPIOC->ODR = (1 << i); // Shift the 1 to left (PC0 to PC3)
odr_value = GPIOC->ODR; // Copy ODR value to variable
delay();
}
}
}
Command
Running with Code Size Limit: 32K
Load "D:\\Objects\\HelloWorld.axf"
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x40000000,0x400FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
WS 1, `odr_value
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Watch 1
Name
Value
Type
odr_value
0x00000002
uint
<Enter expression>
Search


D:\Hello\HelloWorld.uvprojx - µVision [Non-Commercial Use License]
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Disassembly
5: for (volatile int i = 0; i < 4; i++);
0x080003EC 9000 STR r0,[sp,#0x00]
0x080003EE E7FF B 0x080003F0
0x080003F0 9800 LDR r0,[sp,#0x00]
0x080003F2 2803 CMP r0,#0x03
0x080003F4 DC05 BGT 0x08000402
Hello.c*
startup_stm32f10x_ld.s
3 void delay(void)
4 {
5 for (volatile int i = 0; i < 50000; i++);
6 }
7
8 int main(void)
9 {
10 // 1. Enable GPIOC clock
11 RCC->APB2ENR |= (1 << 4);
12
13 // 2. Configure PC0, PC1, PC2, PC3 as output
14 GPIOC->CRL &= ~(0xFFFF << 0); // Clear bits for pin 0-3
15 GPIOC->CRL |= (0x1111 << 0); // Output mode for pin 0-3
16
17 uint32_t odr_value = 0; // Variable to watch
18
19 while (1)
20 {
21 for (int i = 0; i < 4; i++)
22 {
23 GPIOC->ODR = (1 << i); // Shift the 1 to left (PC0 to PC3)
24 odr_value = GPIOC->ODR; // Copy ODR value to variable
25 delay();
26 }
27 }
28 }
Command
Running with Code Size Limit: 32K
Load "D:\\Objects\\HelloWorld.axf"
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x40000000,0x400FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
WS 1, `odr_value
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Watch 1
Name Value Type
odr_value 0x00000004 uint
<Enter expression>
Search
6:34 PM
9/29/2026

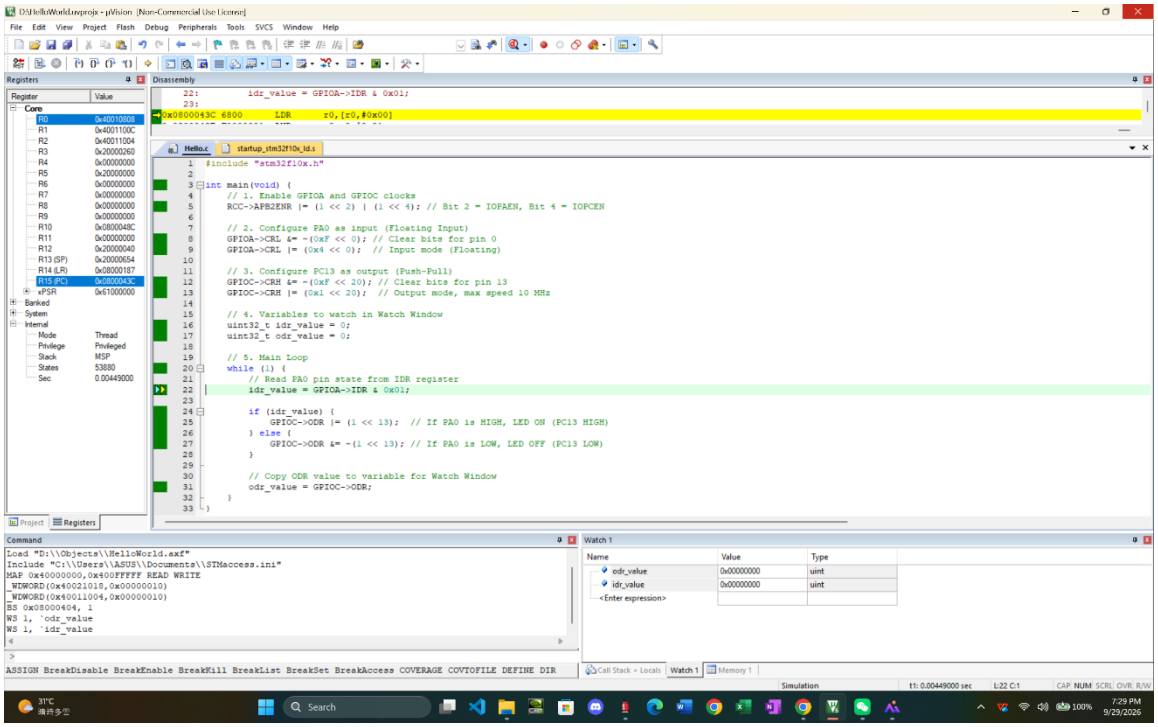

D:\Hello\HelloWorld.uvprojx - µVision [Non-Commercial Use License]
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Disassembly
5: for (volatile int i = 0; i < 4; i++);
0x080003EC 9000 STR r0,[sp,#0x00]
0x080003EE E7FF B 0x080003F0
0x080003F0 9800 LDR r0,[sp,#0x00]
0x080003F2 2803 CMP r0,#0x03
0x080003F4 DC05 BGT 0x08000402
Hello.c*
startup_stm32f10x_ld.s
3 void delay(void)
4 {
5 for (volatile int i = 0; i < 50000; i++);
6 }
7
8 int main(void)
9 {
10 // 1. Enable GPIOC clock
11 RCC->APB2ENR |= (1 << 4);
12
13 // 2. Configure PC0, PC1, PC2, PC3 as output
14 GPIOC->CRL &= ~(0xFFFF << 0); // Clear bits for pin 0-3
15 GPIOC->CRL |= (0x1111 << 0); // Output mode for pin 0-3
16
17 uint32_t odr_value = 0; // Variable to watch
18
19 while (1)
20 {
21 for (int i = 0; i < 4; i++)
22 {
23 GPIOC->ODR = (1 << i); // Shift the 1 to left (PC0 to PC3)
24 odr_value = GPIOC->ODR; // Copy ODR value to variable
25 delay();
26 }
27 }
28 }
Command
Running with Code Size Limit: 32K
Load "D:\\Objects\\HelloWorld.axf"
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x40000000,0x400FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
WS 1, `odr_value
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Watch 1
Name Value Type
odr_value 0x00000008 uint
<Enter expression>
Search
6:44 PM
9/29/2026

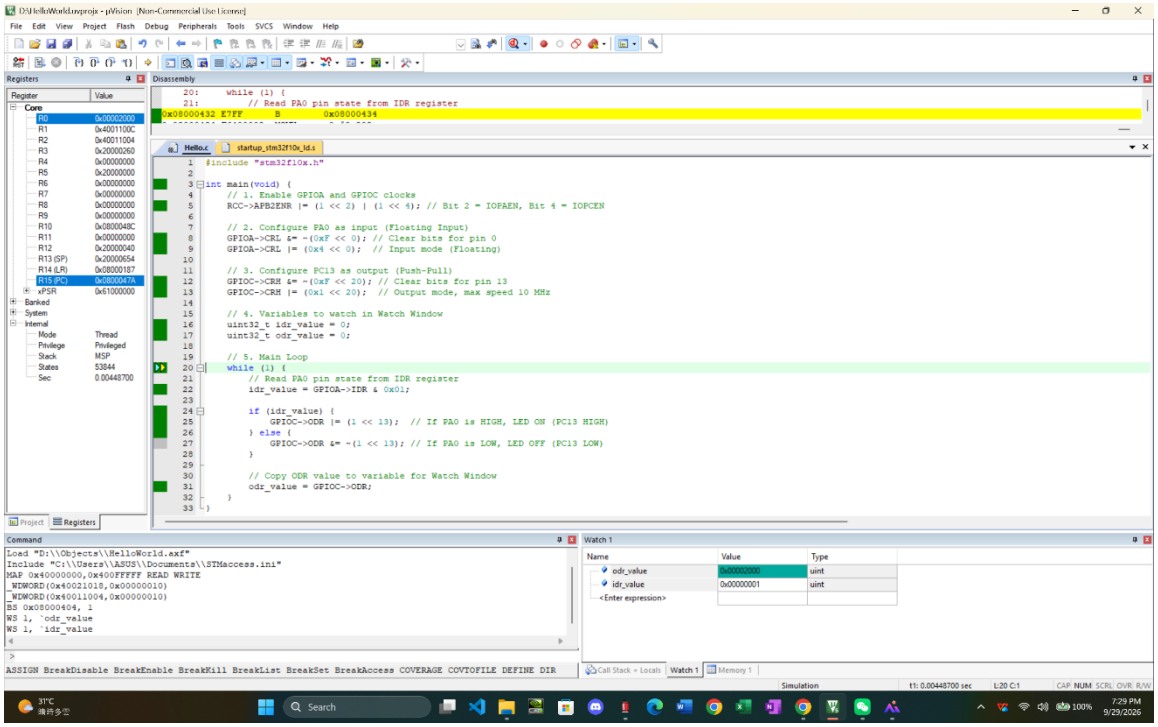
◇ Task 3: Button-Controlled LED (Input Mode Simulation)

Objective:
To simulate reading an input pin (PA0 as a button) using the IDR register and controlling an output pin (PC13 as an LED) using the ODR register.

Description:
In this task, PA0 was configured as a floating input and PC13 as a push-pull output. Inside the main loop, the state of PA0 was read using GPIOA->IDR & 0x01 and stored in the input variable. An if-else condition was used: when input = 1 (button pressed), the 13th bit of GPIOC->ODR was set HIGH to turn the LED ON. When input = 0 (button released), the bit was cleared to turn the LED OFF. The output variable was used to observe the ODR value in the Watch Window.

Output:



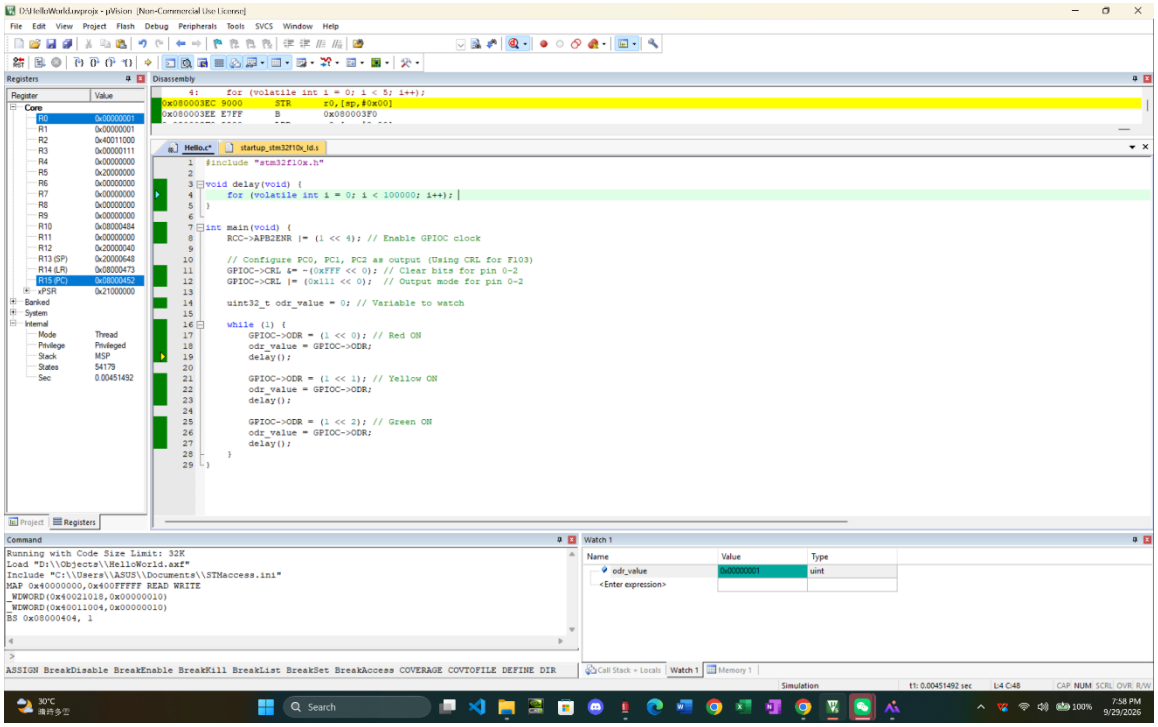


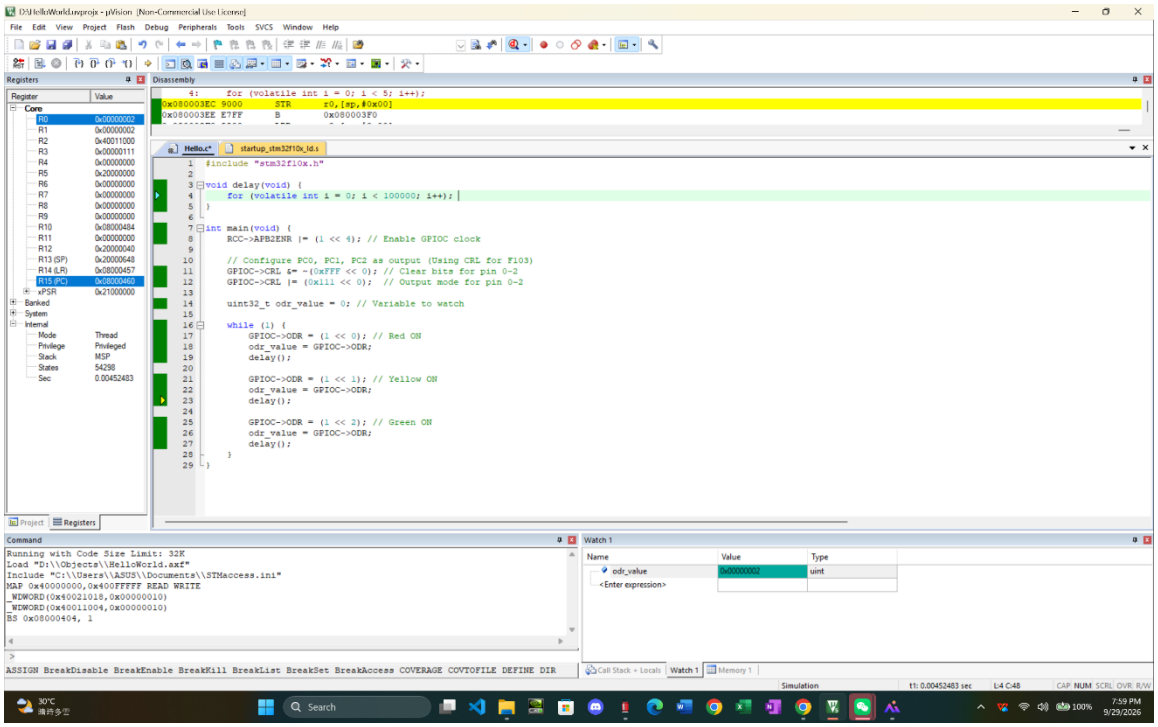
◇ Task 4: Software Traffic Light Controller (State Logic Simulation)

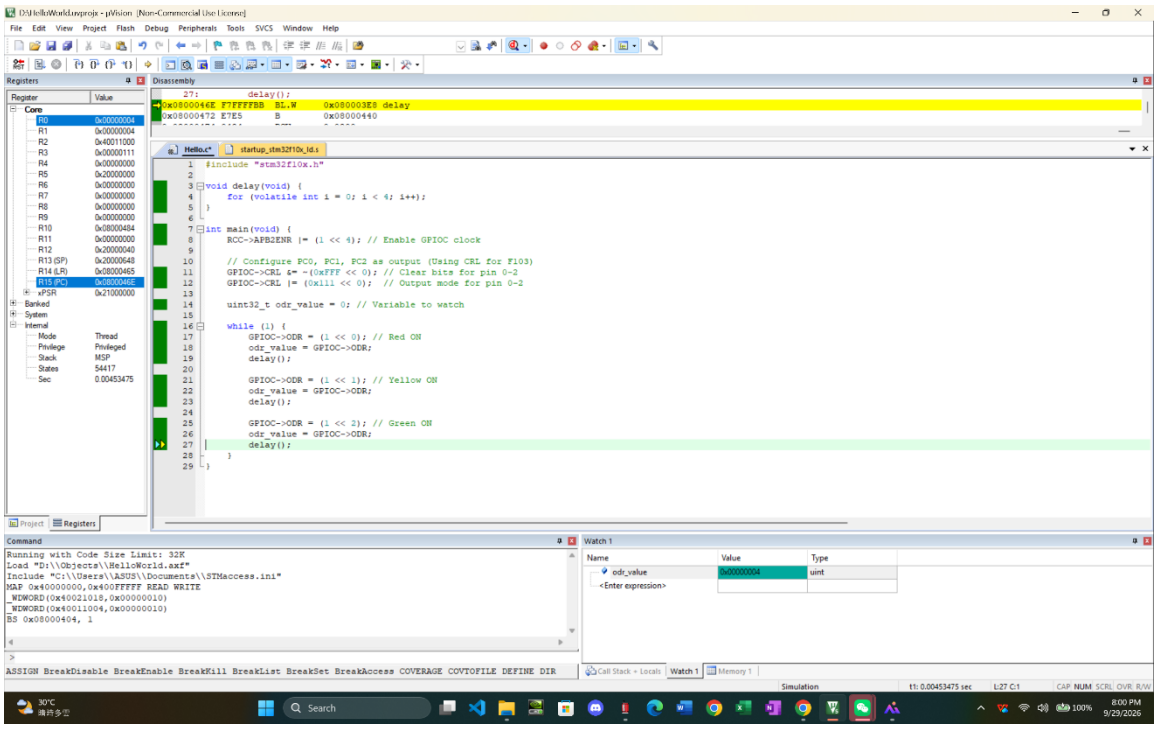
Objective:
To simulate a simple traffic light sequence using software delays and GPIO output pins (PC0, PC1, PC2).

Description:
In this task, PC0 was assigned as Red, PC1 as Yellow, and PC2 as Green. Inside an infinite loop, each pin was turned ON one after another using the ODR register with a delay in between. The output variable was used to monitor the ODR values, which changed as 0x00000001 (Red) → 0x00000002 (Yellow) → 0x00000004 (Green), and then repeated. This created a cyclic traffic light sequence.

Output:


#include "stm32f10x.h"
void delay(void) {
for (volatile int i = 0; i < 100000; i++);
}
int main(void) {
RCC->APB2ENR |= (1 << 4); // Enable GPIOC clock
// Configure PC0, PC1, PC2 as output (Using CRL for F103)
GPIOC->CRL &= ~(0xFFF << 0); // Clear bits for pin 0-2
GPIOC->CRL |= (0x111 << 0); // Output mode for pin 0-2
uint32_t odr_value = 0; // Variable to watch
while (1) {
GPIOC->ODR = (1 << 0); // Red ON
odr_value = GPIOC->ODR;
delay();
GPIOC->ODR = (1 << 1); // Yellow ON
odr_value = GPIOC->ODR;
delay();
GPIOC->ODR = (1 << 2); // Green ON
odr_value = GPIOC->ODR;
delay();
}
}
Running with Code Size Limit: 32K
Load "D:\\Objects\\HelloWorld.axf"
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x40000000,0x400FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
odr_value
0x00000001
uint
Search

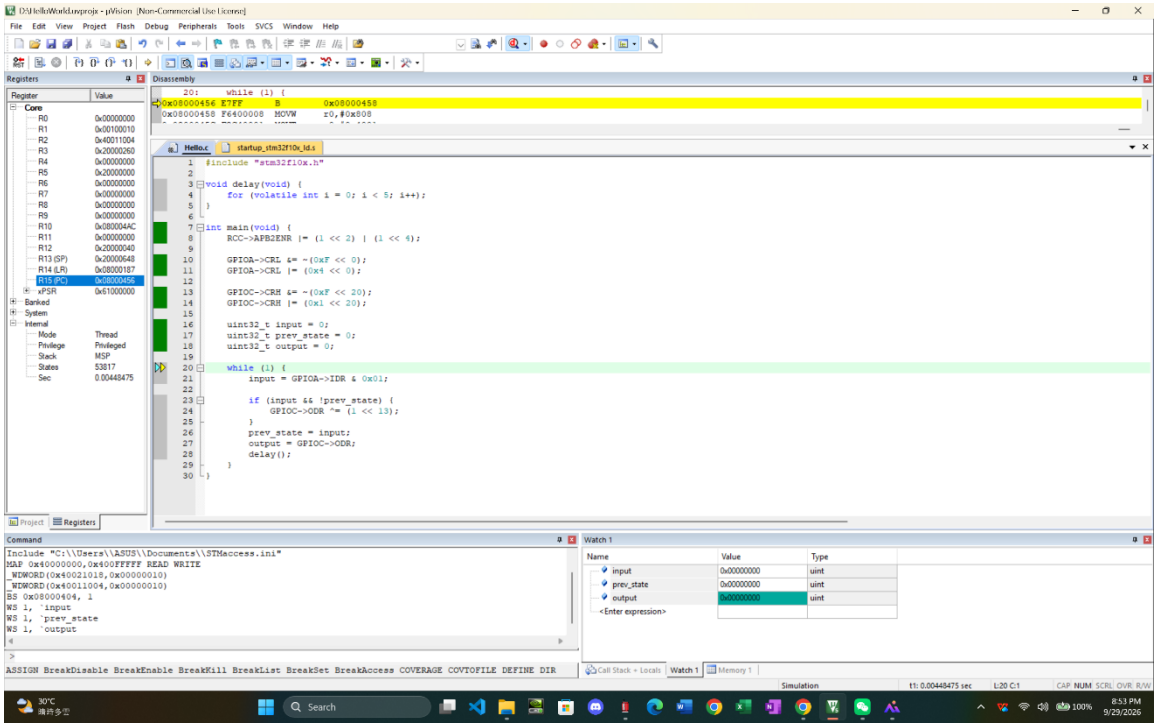

#include "stm32f10x.h"
void delay(void) {
for (volatile int i = 0; i < 100000; i++);
}
int main(void) {
RCC->APB2ENR |= (1 << 4); // Enable GPIOC clock
// Configure PC0, PC1, PC2 as output (Using CRL for F103)
GPIOC->CRL &= ~(0xFFF << 0); // Clear bits for pin 0-2
GPIOC->CRL |= (0x111 << 0); // Output mode for pin 0-2
uint32_t odr_value = 0; // Variable to watch
while (1) {
GPIOC->ODR = (1 << 0); // Red ON
odr_value = GPIOC->ODR;
delay();
GPIOC->ODR = (1 << 1); // Yellow ON
odr_value = GPIOC->ODR;
delay();
GPIOC->ODR = (1 << 2); // Green ON
odr_value = GPIOC->ODR;
delay();
}
}
Running with Code Size Limit: 32K
Load "D:\\Objects\\HelloWorld.axf"
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x40000000,0x400FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
odr_value
0x00000002
uint
Search

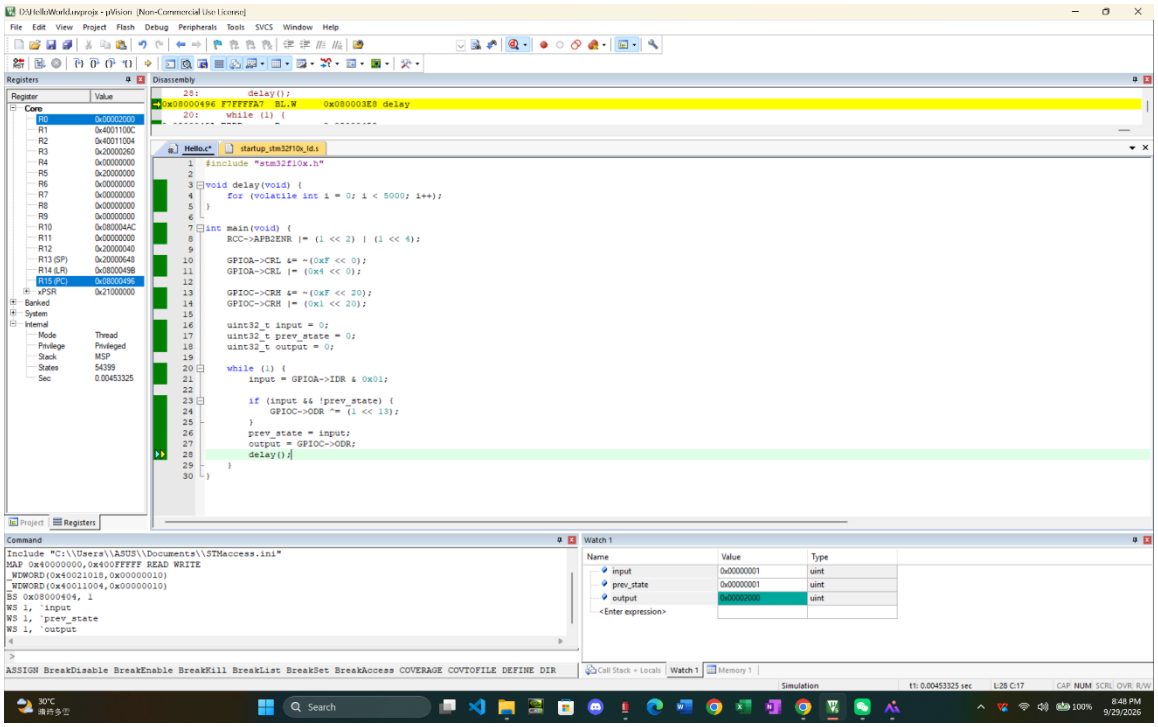


◇ Task 5: Edge Detection (Polling Simulation)

Objective:
To detect a rising edge (0 → 1 transition) in an input signal (PA0) through software polling using the IDR register and to toggle an output LED (PC13) only on that edge.

Description:
In this task, PA0 was configured as input and PC13 as output. Two variables, input (current state) and prev_state (previous state), were used for polling. The LED was toggled only when input = 1 and prev_state = 0 (a rising edge). When the button was held pressed (input = 1, prev_state = 1), the condition became false and the LED did not toggle again. This confirmed edge-triggered behavior, where the LED toggles once per button press. The output variable was used to verify the ODR value in the Watch Window.


D:\Hello\HelloWorld.uvprojx - µVision (Non-Commercial Use License)
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Disassembly
Hello.c
startup_stm32f10x_ld.s
#include "stm32f10x.h"
void delay(void) {
for (volatile int i = 0; i < 5; i++);
}
int main(void) {
RCC->APB2ENR |= (1 << 2) | (1 << 4);
GPIOA->CRL &= ~(0xF << 0);
GPIOA->CRL |= (0x4 << 0);
GPIOC->CRH &= ~(0xF << 20);
GPIOC->CRH |= (0x1 << 20);
uint32_t input = 0;
uint32_t prev_state = 0;
uint32_t output = 0;
while (1) {
input = GPIOA->IDR & 0x01;
if (input && !prev_state) {
GPIOC->ODR ^= (1 << 13);
}
prev_state = input;
output = GPIOC->ODR;
delay();
}
}
Command
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x60000000,0x600FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
WS 1, `input
WS 1, `prev_state
WS 1, `output
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Watch 1
Name Value Type
input 0x00000000 uint
prev_state 0x00000000 uint
output 0x00000000 uint
<Enter expression>
Simulation
t1: 0.00448475 sec
L:20 C:1
8:53 PM
9/29/2026


D:\Hello\HelloWorld.uvprojx - µVision (Non-Commercial Use License)
File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
Registers
Disassembly
Hello.c*
startup_stm32f10x_ld.s
#include "stm32f10x.h"
void delay(void) {
for (volatile int i = 0; i < 5000; i++);
}
int main(void) {
RCC->APB2ENR |= (1 << 2) | (1 << 4);
GPIOA->CRL &= ~(0xF << 0);
GPIOA->CRL |= (0x4 << 0);
GPIOC->CRH &= ~(0xF << 20);
GPIOC->CRH |= (0x1 << 20);
uint32_t input = 0;
uint32_t prev_state = 0;
uint32_t output = 0;
while (1) {
input = GPIOA->IDR & 0x01;
if (input && !prev_state) {
GPIOC->ODR ^= (1 << 13);
}
prev_state = input;
output = GPIOC->ODR;
delay();
}
}
Command
Include "C:\\Users\\ASUS\\Documents\\STMaccess.ini"
MAP 0x60000000,0x600FFFFF READ WRITE
_WDWORD(0x40021018,0x00000010)
_WDWORD(0x40011004,0x00000010)
BS 0x08000404, 1
WS 1, `input
WS 1, `prev_state
WS 1, `output
ASSIGN BreakDisable BreakEnable BreakKill BreakList BreakSet BreakAccess COVERAGE COVTOFILE DEFINE DIR
Watch 1
Name Value Type
input 0x00000001 uint
prev_state 0x00000001 uint
output 0x00002000 uint
<Enter expression>
Simulation
t1: 0.00453325 sec
L:28 C:17
8:48 PM
9/29/2026

Conclusion:

In this lab, I successfully completed all five tasks related to General Purpose Input/Output (GPIO) programming on the STM32F103 microcontroller using Keil uVision simulation. Through these tasks, I gained practical knowledge of register-level programming, including how to enable peripheral clocks, configure GPIO pins as input or output, and manipulate data using the IDR (Input Data Register) and ODR (Output Data Register).

I learned how to toggle a single pin for LED blinking, shift bits to create a running light pattern, read an input pin to control an output, generate a sequential traffic light pattern, and detect a rising edge to achieve edge-triggered behavior. Additionally, I became familiar with the Keil Debugger, Watch Window, and Breakpoints, which helped me monitor variable values and observe real-time changes in the simulation.

Overall, this lab strengthened my understanding of embedded system fundamentals, bit manipulation, and hardware-software interaction, which will be highly valuable for future microcontroller-based projects.